

HW3: Part I

Part 1.1 - Parsing a page with BeautifulSoup

1. Use the requests library to get the following web page and store the results in a variable called "page".

<http://dataquestio.github.io/web-scraping-pages/simple.html>

```
In [1]: import requests
page = requests.get("http://dataquestio.github.io/web-scraping-pages/simple.html")
page
```

Out[1]: <Response [200]>

2. View the `status_code` and `content` attributes of `page`.

```
In [2]: page.status_code
```

Out[2]: 200

```
In [3]: page.content
```

Out[3]: b'<!DOCTYPE html>\n<html>\n <head>\n <title>A simple example page</title>\n </head>\n <body>\n <p>Here is some simple content for this page.</p>\n </body>\n</html>'

3. Parse `page.content` using BeautifulSoup and the 'html.parser', storing the result in the variable `soup`.
Display the results using `soup`'s `prettify` method by typing `print(soup.prettify())`.

```
In [4]: from bs4 import BeautifulSoup
soup = BeautifulSoup(page.content, 'html.parser')
print(soup.prettify())
```

```
<!DOCTYPE html>
<html>
  <head>
    <title>
      A simple example page
    </title>
  </head>
  <body>
    <p>
      Here is some simple content for this page.
    </p>
  </body>
</html>
```

4. Check out the `children` attribute of `soup`. Since this attribute is a Python generator, wrap it inside a `list` command to set all the contents at once.

```
In [5]: list(soup.children)
```

```
Out[5]: ['html',
        '\n',
        <html>
        <head>
        <title>A simple example page</title>
        </head>
        <body>
        <p>Here is some simple content for this page.</p>
        </body>
        </html>]
```

5. Report the type of each child.

```
In [6]: {item:type(item) for item in list(soup.children)}
```

```
Out[6]: {'html': bs4.element.Doctype,
        '\n': bs4.element.NavigableString,
        <html>
        <head>
        <title>A simple example page</title>
        </head>
        <body>
        <p>Here is some simple content for this page.</p>
        </body>
        </html>: bs4.element.Tag}
```

6. Let's focus on the last child since it contains the html. Store this child in a variable called `html`. You may need to use the `list` command again to access this item. Display `html`.

```
In [7]: html = list(soup.children)[2]
        html
```

```
Out[7]: <html>
        <head>
        <title>A simple example page</title>
        </head>
        <body>
        <p>Here is some simple content for this page.</p>
        </body>
        </html>
```

7. This child has children as well. Display `html`'s children.

```
In [8]: list(html.children)
```

```
Out[8]: ['\n',
<head>
<title>A simple example page</title>
</head>,
'\n',
<body>
<p>Here is some simple content for this page.</p>
</body>,
'\n']
```

8. Store the child that has the string "Here is some simple content..." in a variable called `body` . Display `body` .

```
In [9]: body = list(html.children)[3]
body
```

```
Out[9]: <body>
<p>Here is some simple content for this page.</p>
</body>
```

9. Use the `.get_text()` method of BeautifulSoup objects (including children) to extract the text from this child and store it in a variable called `body_text` .

```
In [10]: body_text = body.get_text()
body_text
```

```
Out[10]: '\nHere is some simple content for this page.\n'
```

10. Remove the newline characters using the strip command and display the result.

```
In [11]: body_text.strip()
```

```
Out[11]: 'Here is some simple content for this page.'
```

Part 1.2 Working with Tags

Finding all instances of a tag at once

1. Use BeautifulSoup's `find_all` method to find all `p` elements within the `soup` object found above and display the results.

```
In [12]: soup.find_all('p')
```

```
Out[12]: [<p>Here is some simple content for this page.</p>]
```

2. How many instances of `p` were found?

3. Repeat #1 but use the find method instead. What is the difference in the data type of the result?

```
In [13]: soup.find('p')
```

```
Out[13]: <p>Here is some simple content for this page.</p>
```

We have a single result instead of a list.

Searching for tags by class and id

4. Get the following web page using `requests` storing the result in the variable `page`.

http://dataquestio.github.io/web-scraping-pages/ids_and_classes.html

Then parse it as you parsed the page in the previous part, storing the bs4 object in the variable `soup`.

Display `soup` with `prettify` as above.

```
In [14]: page = requests.get("http://dataquestio.github.io/web-scraping-pages/ids_and_classes.html")
soup = BeautifulSoup(page.content, 'html.parser')
print(soup.prettify())
```

```
<html>
<head>
  <title>
    A simple example page
  </title>
</head>
<body>
  <div>
    <p class="inner-text first-item" id="first">
      First paragraph.
    </p>
    <p class="inner-text">
      Second paragraph.
    </p>
  </div>
  <p class="outer-text first-item" id="second">
    <b>
      First outer paragraph.
    </b>
  </p>
  <p class="outer-text">
    <b>
      Second outer paragraph.
    </b>
  </p>
</body>
</html>
```

5. Use BeautifulSoup's `find_all` method to find all `p` elements within the `soup` object that are of the class `'outer-text'`. To do this, supply the `class_` parameter to `find_all` with value of `'outer-text'`. Display the results.

```
In [15]: outer_text_ps = soup.find_all('p', class_='outer-text')
outer_text_ps
```

```
Out[15]: [<p class="outer-text first-item" id="second">
  <b>
    First outer paragraph.
  </b>
</p>,
<p class="outer-text">
  <b>
    Second outer paragraph.
  </b>
</p>]
```

6. How many elements were found?

```
In [16]: print(f'{len(outer_text_ps)} elements were found.')
```

2 elements were found.

7. Would `find_all` work without a tag name (e.g., no `p`) but only a class. Try it and answer.

```
In [17]: soup.find_all(class_="outer-text")
```

```
Out[17]: [<p class="outer-text first-item" id="second">
  <b>
    First outer paragraph.
  </b>
</p>,
<p class="outer-text">
  <b>
    Second outer paragraph.
  </b>
</p>]
```

Yes, it returns all elements with that class regardless of tag.

8. Another attribute besides `class_` is `id`. Run `find_all` on `soup` to find all elements with `id` value of `"first"` and display the results.

```
In [18]: soup.find_all(id="first")
```

```
Out[18]: [<p class="inner-text first-item" id="first">
  First paragraph.
</p>]
```

Using CSS Selectors

CSS Selectors allow you to extract information in more complicated ways. For instance, you can combine tags in a search.

9. Type `soup.select('div p')` and explain what the results are. You can refer back to the full html you output above.

```
In [19]: soup.select("div p")
```

```
Out[19]: [<p class="inner-text first-item" id="first">
          First paragraph.
        </p>,
        <p class="inner-text">
          Second paragraph.
        </p>]
```

This found only the `p` elements within `div` tags.

So you should realize we have various ways of extracting information from a web page by using attributes in its html tags.

Part 1.3 Web Scraping Weather Data

We will download and web scrape weather data.

1. Run the following code. Explain what `forecast_items` contains.

```
In [20]: page = requests.get("http://forecast.weather.gov/MapClick.php?lat=37.7772&lon=-122.4168")
soup = BeautifulSoup(page.content, 'html.parser')
seven_day = soup.find(id="seven-day-forecast")
forecast_items = seven_day.find_all(class_="tombstone-container")
forecast_items
```

```

Out[20]: [<div class="tombstone-container">
  <p class="period-name">This<br/>Afternoon</p>
  <p></p><p class="short-desc">Sunny</p><p class="temp temp-high">High: 69 °F</p></div>,
  <div class="tombstone-container">
  <p class="period-name">Tonight<br/><br/></p>
  <p></p><p class="short-desc">Partly Cloudy</p><p class="temp temp-low">Low: 56 °F</p></div>,
  <div class="tombstone-container">
  <p class="period-name">Wednesday<br/><br/></p>
  <p></p><p class="short-desc">Mostly Sunny</p><p class="temp temp-high">High: 70 °F</p></div>,
  <div class="tombstone-container">
  <p class="period-name">Wednesday<br/>Night</p>
  <p></p><p class="short-desc">Partly Cloudy</p><p class="temp temp-low">Low: 56 °F</p></div>,
  <div class="tombstone-container">
  <p class="period-name">Thursday<br/><br/></p>
  <p></p><p class="short-desc">Mostly Sunny</p><p class="temp temp-high">High: 70 °F</p></div>,
  <div class="tombstone-container">
  <p class="period-name">Thursday<br/>Night</p>
  <p></p><p class="short-desc">Partly Cloudy</p><p class="temp temp-low">Low: 57 °F</p></div>,
  <div class="tombstone-container">
  <p class="period-name">Friday<br/><br/></p>
  <p></p><p class="short-desc">Mostly Sunny</p><p class="temp temp-high">High: 71 °F</p></div>,
  <div class="tombstone-container">
  <p class="period-name">Friday<br/>Night</p>
  <p></p><p class="short-desc">Mostly Clear</p><p class="temp temp-low">Low: 56 °F</p></div>,
  <div class="tombstone-container">
  <p class="period-name">Saturday<br/><br/></p>
  <p></p><p class="short-desc">Sunny</p><p class="temp temp-high">High: 73 °F</p></div>]

```

On the web page, these are the rectangular areas (shaped roughly like 'tombstones') that contain the day/time, weather condition, graphic, and temperature for tonight and the next four days.

2. Display one of these tombstones using `prettify`. (Hint - it is one of the list items found in the previous question.)

```
In [21]: print(forecast_items[2].prettify())
```

```
<div class="tombstone-container">
  <p class="period-name">
    Wednesday
  <br/>
  <br/>
</p>
<p>
  
</p>
<p class="short-desc">
  Mostly Sunny
</p>
<p class="temp temp-high">
  High: 70 °F
</p>
</div>
```

3. Pick a tombstone and store it in the variable `cur_tombstone`. Use the `find` method with the `class_` attribute applied to this tombstone. Do this three times with `class_` set to 'period-name', 'short-descr', and 'temp', in turn and save the results in variables `period`, `short_desc`, and `temp`, respectively. Add `.get_text()` to the end of each command. Print these three variables.

```
In [22]: cur_tombstone = forecast_items[0]
period = cur_tombstone.find(class_="period-name").get_text()
short_desc = cur_tombstone.find(class_="short-desc").get_text()
temp = cur_tombstone.find(class_="temp").get_text()

print(period)
print(short_desc)
print(temp)
```

```
ThisAfternoon
Sunny
High: 69 °F
```

4. There is an image in the tombstone. Images in web pages often have alternate text explaining what the image is. Find the image using the `find` method applied to `cur_tombstone` searching for the `img` tag, then access its alt text through the `alt` attribute. You can do the latter with a dictionary-like call `...['alt']`.

```
In [23]: cur_tombstone.find("img")['alt']
```

```
Out[23]: 'This Afternoon: Sunny, with a high near 69. West southwest wind 6 to 11 mph. '
```

Next we will extract all of the 7-day forecast info and store it in a dataframe.

```
In [24]: period_tags = seven_day.select(".tombstone-container .period-name")
periods = [pt.get_text() for pt in period_tags]
```

```

short_descs = [sd.get_text() for sd in seven_day.select(".tombstone-container .short-desc")]
temps = [t.get_text() for t in seven_day.select(".tombstone-container .temp")]
descs = [d["title"] for d in seven_day.select(".tombstone-container img")]

print(periods, '\n')
print(short_descs, '\n')
print(temps, '\n')
print(descs, '\n')

```

```

['ThisAfternoon', 'Tonight', 'Wednesday', 'WednesdayNight', 'Thursday', 'ThursdayNight', 'Friday', 'FridayNight', 'Saturday']

```

```

['Sunny', 'Partly Cloudy', 'Mostly Sunny', 'Partly Cloudy', 'Mostly Sunny', 'Partly Cloudy', 'Mostly Sunny', 'Mostly Clear', 'Sunny']

```

```

['High: 69 °F', 'Low: 56 °F', 'High: 70 °F', 'Low: 56 °F', 'High: 70 °F', 'Low: 57 °F', 'High: 71 °F', 'Low: 56 °F', 'High: 73 °F']

```

```

['This Afternoon: Sunny, with a high near 69. West southwest wind 6 to 11 mph. ', 'Tonight: Partly cloudy, with a low around 56. Southwest wind 7 to 11 mph. ', 'Wednesday: Mostly sunny, with a high near 70. Southwest wind 7 to 13 mph. ', 'Wednesday Night: Partly cloudy, with a low around 56. West southwest wind 6 to 14 mph, with gusts as high as 18 mph. ', 'Thursday: Mostly sunny, with a high near 70. Southwest wind 6 to 11 mph. ', 'Thursday Night: Partly cloudy, with a low around 57.', 'Friday: Mostly sunny, with a high near 71.', 'Friday Night: Mostly clear, with a low around 56.', 'Saturday: Sunny, with a high near 73.']

```

```

In [25]: import pandas as pd
weather = pd.DataFrame({
    "period": periods,
    "short_desc": short_descs,
    "temp": temps,
    "desc": descs
})
weather

```

```

Out[25]:

```

	period	short_desc	temp	desc
0	ThisAfternoon	Sunny	High: 69 °F	This Afternoon: Sunny, with a high near 69. We...
1	Tonight	Partly Cloudy	Low: 56 °F	Tonight: Partly cloudy, with a low around 56. ...
2	Wednesday	Mostly Sunny	High: 70 °F	Wednesday: Mostly sunny, with a high near 70. ...
3	WednesdayNight	Partly Cloudy	Low: 56 °F	Wednesday Night: Partly cloudy, with a low aro...
4	Thursday	Mostly Sunny	High: 70 °F	Thursday: Mostly sunny, with a high near 70. S...
5	ThursdayNight	Partly Cloudy	Low: 57 °F	Thursday Night: Partly cloudy, with a low arou...
6	Friday	Mostly Sunny	High: 71 °F	Friday: Mostly sunny, with a high near 71.
7	FridayNight	Mostly Clear	Low: 56 °F	Friday Night: Mostly clear, with a low around 56.
8	Saturday	Sunny	High: 73 °F	Saturday: Sunny, with a high near 73.

The next block of code extracts the numeric values from the temperature column.

```
In [26]: temp_nums = weather["temp"].str.extract("(?P<temp_num>\d+)", expand=False)
weather["temp_num"] = temp_nums.astype('int')
temp_nums
```

```
Out[26]: 0    69
1    56
2    70
3    56
4    70
5    57
6    71
7    56
8    73
Name: temp_num, dtype: object
```

5. What data type is `temp_nums` ?

Pandas Series

6. Find the mean temperature using `temp_nums` .

```
In [27]: print(f'The mean temperature is {weather["temp_num"].mean():.2f}.')
```

The mean temperature is 64.22.

7. Write the code necessary to extract the information from the weather web page shown in the green box below and store it in a dictionary or dataframe. Note: your information will look different than the one shown as I took this screenshot at a different time.

 Current conditions - middle column

```
In [28]: table = soup.find('table')
table
```

```
Out[28]: <table>
<tr>
<td class="text-right"><b>Humidity</b></td>
<td>80%</td>
</tr>
<tr>
<td class="text-right"><b>Wind Speed</b></td>
<td>NA NA MPH</td>
</tr>
<tr>
<td class="text-right"><b>Barometer</b></td>
<td>NA</td>
</tr>
<tr>
<td class="text-right"><b>Dewpoint</b></td>
<td>54°F (12°C)</td>
</tr>
<tr>
<td class="text-right"><b>Visibility</b></td>
<td>NA</td>
</tr>
<tr>
<td class="text-right"><b>Last update</b></td>
<td>
05 Sep 10:43 AM PDT
</td>
</tr>
</table>
```

```
In [29]: tds = table.find_all('td')
tds
```

```
Out[29]: [<td class="text-right"><b>Humidity</b></td>,
<td>80%</td>,
<td class="text-right"><b>Wind Speed</b></td>,
<td>NA NA MPH</td>,
<td class="text-right"><b>Barometer</b></td>,
<td>NA</td>,
<td class="text-right"><b>Dewpoint</b></td>,
<td>54°F (12°C)</td>,
<td class="text-right"><b>Visibility</b></td>,
<td>NA</td>,
<td class="text-right"><b>Last update</b></td>,
<td>
05 Sep 10:43 AM PDT
</td>]
```

Headers alternate with values. And we will set headers as keys, values as values, in a dictionary.

```
In [30]: weather_dict = {}
for i in range(0, len(tds), 2):
    key = tds[i].get_text()
    value = tds[i+1].get_text()
    weather_dict[key] = value
weather_dict
```

```
Out[30]: {'Humidity': '80%',  
          'Wind Speed': 'NA NA MPH',  
          'Barometer': 'NA',  
          'Dewpoint': '54°F (12°C)',  
          'Visibility': 'NA',  
          'Last update': '\n  
05 Sep 10:43 AM PDT  
'}
```